En el programa incorporamos funciones de la biblioteca estándar de C para abrir, leer, escribir, verificar y cerrar archivos binarios. También desarrollamos funciones propias que combinan esas operaciones para implementar la persistencia de los usuarios.

Vamos a distinguir ambos grupos y explicar cómo trabajan en nuestro CRUD.

1. ¿Qué es FILE*?

Antes de estudiar las funciones, necesitamos entender qué representa un archivo abierto en C.

FILE *archivo;

- FILE es un tipo de dato definido en <stdio.h> que representa un flujo de datos.
- archivo es un puntero a ese tipo de dato.
- A través de este puntero podemos realizar operaciones de lectura y escritura sobre el archivo abierto.

Importante: declarar FILE *archivo no abre ningún archivo. Solamente declara la variable que utilizaremos para trabajar con él.

2. Funciones de la biblioteca estándar

fopen() — Abrir un archivo

Permite abrir un archivo existente o crear uno nuevo, dependiendo del modo de apertura.

Sintaxis:

FILE *fopen(const char *ruta, const char *modo);

Recibe dos argumentos: la ruta del archivo y el modo en que deseamos abrirlo. Devuelve un FILE* si pudo abrirlo o NULL si ocurrió un error.

En nuestro programa utilizamos dos modos:

Modo	Significado	Comportamiento
"rb"	Read Binary	Abre un archivo existente para lectura binaria.
"wb"	Write Binary	Crea un archivo para escritura binaria. Si existe, reemplaza su contenido.

Ejemplo de lectura:

```
FILE *archivo = fopen("data/usuarios.dat", "rb");


if (archivo != NULL) {
   puts("Archivo abierto correctamente.");
} else {
   puts("No se pudo abrir el archivo.");
}
```

Hay una diferencia importante: "rb" no crea el archivo si no existe, mientras que "wb" sí puede crearlo.

En nuestro CRUD utilizamos "rb" al iniciar y "wb" al salir.

fwrite() — Escribir datos binarios

Permite escribir uno o varios elementos de memoria en un archivo binario.

Sintaxis:

fwrite(datos, tamanio, cantidad, archivo);

Parámetro	Significado
datos	Dirección de memoria donde comienzan los datos que queremos escribir.
tamanio	Tamaño en bytes de cada elemento.
cantidad	Número de elementos que queremos escribir.
archivo	Puntero al archivo abierto.

La función devuelve la cantidad de elementos completos que pudo escribir.

Ejemplo 1: guardar un entero

int cantidad = 3;

fwrite(&cantidad, sizeof(int), 1, archivo);

Interpretación:

- &cantidad: dirección de memoria del entero.
- sizeof(int): tamaño en bytes de un entero.
- 1: queremos escribir un solo entero.
- archivo: destino de la escritura.

Si el entero ocupa 4 bytes, se intentan escribir esos 4 bytes en el archivo, en su representación binaria.

Ejemplo 2: guardar un array de estructuras

Usuario usuarios[MAX_USUARIOS];

int cantidad = 3;

fwrite(usuarios, sizeof(Usuario), cantidad, archivo);

En este caso escribimos las primeras tres estructuras Usuario del array.

¿Por qué no colocamos &usuarios? Porque el nombre del array, utilizado en esta expresión, ya representa la dirección de su primer elemento.

Representación de la escritura

Esquema simplificado: fwrite copia los bytes de las estructuras desde la RAM hacia el archivo.

Para verificar que se escribieron todos los elementos:

```
int escrituraCorrecta;


escrituraCorrecta =
   fwrite(usuarios, sizeof(Usuario), cantidad, archivo)
   == (size_t)cantidad;
```

La comparación será verdadera (1) si se escribieron los cantidad elementos solicitados.

fread() — Leer datos binarios

Es la operación inversa de fwrite().

Permite recuperar datos binarios almacenados en un archivo y copiarlos a una región de memoria.

Sintaxis:

```
fread(destino, tamanio, cantidad, archivo);
```

Los parámetros tienen el mismo significado que en fwrite(), con una diferencia: ahora el primer argumento indica dónde queremos depositar los datos leídos.

Ejemplo 1: recuperar un entero

```
int cantidad;


fread(&cantidad, sizeof(int), 1, archivo);
```

La función lee los bytes que representan un entero y los deposita en la variable cantidad.

Ejemplo 2: recuperar un array de estructuras

```
Usuario usuarios[MAX_USUARIOS];


fread(usuarios, sizeof(Usuario), cantidad, archivo);
```

La función recupera del archivo cantidad estructuras y las coloca consecutivamente en el array.

Representación de la lectura

fread recupera del archivo los bytes de cada estructura y los copia al array en RAM.

Es indispensable verificar cuántos elementos se leyeron:

```
int lecturaCorrecta;


lecturaCorrecta =
   fread(usuarios, sizeof(Usuario), cantidad, archivo)
   == (size_t)cantidad;
```

Precaución: antes de leer, debemos comprobar que cantidad no supere la capacidad del array. De lo contrario, podríamos sobrescribir memoria que no le corresponde.

fgetc() — Leer un byte del archivo

Normalmente se utiliza para leer caracteres, pero también puede emplearse para leer un byte de un archivo binario.

Sintaxis:

```
int caracter = fgetc(archivo);
```

Devuelve el valor del byte leído, convertido a int, o EOF si se alcanza el final del archivo o se produce un error.

¿Por qué lo usamos en nuestro CRUD?

Después de recuperar la cantidad de usuarios y todas sus estructuras, queremos asegurarnos de que no existan datos adicionales inesperados.

```
int caracterExtra;


caracterExtra = fgetc(archivo);


if (caracterExtra == EOF && !ferror(archivo)) {
   puts("No hay datos adicionales.");
}
```

Si fgetc() encuentra otro byte, significa que existe información adicional después de los registros esperados.

En nuestro ejemplo tratamos esa situación como un archivo de formato inválido.

ferror() — Detectar errores de lectura o escritura

Permite consultar si ocurrió un error durante las operaciones realizadas sobre un archivo.

Sintaxis:

```
int resultado = ferror(archivo);
```

- Devuelve 0 si no se registró ningún error.
- Devuelve un valor distinto de cero si se registró un error.

En nuestro CRUD:

```
if (caracterExtra == EOF && !ferror(archivo)) {
   lecturaCorrecta = 1;
}
```

Esta comprobación es importante porque EOF puede significar dos cosas diferentes: que llegamos al final del archivo o que ocurrió un error de lectura.

ferror() nos permite distinguir ambas situaciones.

fclose() — Cerrar un archivo

Permite cerrar un archivo que fue abierto previamente con fopen().

Sintaxis:

```
fclose(archivo);
```

Cuando terminamos de utilizar un archivo, debemos cerrarlo para liberar los recursos asociados. Además, al cerrar un archivo utilizado para escritura, se intenta completar el envío de los datos pendientes al sistema de archivos.

Devuelve 0 si el cierre fue satisfactorio y EOF si ocurrió un error.

En nuestro ejemplo:

```
int cierreCorrecto;


cierreCorrecto = fclose(archivo) == 0;
```

No alcanza con comprobar que fwrite() haya terminado correctamente: también debemos comprobar el cierre, porque allí podrían detectarse errores de escritura pendientes.

3. Funciones propias que incorporamos al CRUD

Las funciones anteriores pertenecen a la biblioteca estándar de C. Sobre ellas construimos las siguientes funciones propias.

Organización de responsabilidades

main()

Carga inicial

cargarUsuarios()

obtenerCarpeta()

leerUsuariosBinarios()

Utiliza fopen, fread, fgetc, ferror y fclose.

Guardado final

guardarUsuarios()

prepararCarpeta()

escribirUsuariosBinarios()

Utiliza fopen, fwrite y fclose.

prepararCarpeta() también utiliza obtenerCarpeta().

leerUsuariosBinarios()

```
int leerUsuariosBinarios(
   FILE *archivo,
   Usuario usuarios[],
   int *cantidad
);
```

Recibe un archivo que ya está abierto y realiza la lectura.

Su trabajo consiste en recuperar primero el entero que representa la cantidad de usuarios, validar que esté entre 0 y MAX_USUARIOS, leer exactamente esa cantidad de estructuras y comprobar que el archivo no contenga bytes sobrantes.

Solamente si todo fue correcto actualiza *cantidad y devuelve 1. En caso contrario devuelve 0.

cargarUsuarios()

```
int cargarUsuarios(
   Usuario usuarios[],
   int *cantidad
);
```

Es la función que coordina toda la carga al iniciar el programa.

Construye la ruta data/usuarios.dat, intenta abrir el archivo en modo "rb" y, si lo consigue, llama a leerUsuariosBinarios().

Si el archivo todavía no existe, comienza con cantidad = 0. Si existe pero no se puede leer correctamente, informa el error.

escribirUsuariosBinarios()

```
int escribirUsuariosBinarios(
   FILE *archivo,
   const Usuario usuarios[],
   int cantidad
);
```

Recibe un archivo ya abierto para escritura.

Su responsabilidad es guardar primero cantidad mediante fwrite() y después escribir las estructuras del array.

Devuelve 1 si las escrituras se completaron y 0 si alguna falló. No necesita modificar el array, por eso recibe const Usuario usuarios[].

guardarUsuarios()

```
int guardarUsuarios(
   const Usuario usuarios[],
   int cantidad
);
```

Coordina la persistencia cuando el usuario elige salir del programa.

Primero prepara la carpeta data, después abre usuarios.dat en modo "wb", llama a escribirUsuariosBinarios() y finalmente cierra el archivo verificando el resultado.

obtenerCarpeta() y prepararCarpeta()

Estas funciones no manipulan directamente el contenido binario, sino que ayudan a localizar y preparar las carpetas.

obtenerCarpeta() construye una ruta a partir de la ubicación del ejecutable y del nombre de una carpeta, por ejemplo data.

prepararCarpeta() intenta crear esa carpeta con _mkdir() si todavía no existe.

En nuestro programa se reutilizan también para los reportes de texto. Esta implementación de rutas es específica de Windows.

4. Ejemplo completo: ¿qué sucede con tres usuarios?

Supongamos que tenemos tres usuarios cargados en memoria.

Simulador conceptual de lectura y escritura

Guardar (fwrite)Cargar (fread)

Desde la RAM hacia usuarios.dat

Array en RAM

usuarios[0]

Ana

usuarios[1]

Luis

usuarios[2]

Eva

fwrite()

Archivo usuarios.dat

cantidad = 3

Usuario Ana

Usuario Luis

Usuario Eva

Primero se escribe el número 3 y después los tres registros completos.

Los nombres se muestran para ilustrar los registros. En el archivo real se guardan sus bytes, no estas etiquetas.

5. Cuadro resumen para estudiar

Elemento	**¿Para qué lo usamos?**
FILE*	Representar un archivo abierto.
fopen()	Abrir o crear el archivo.
fwrite()	Escribir datos de RAM al archivo.
fread()	Leer datos del archivo hacia RAM.
fgetc()	Comprobar si queda algún byte adicional.
ferror()	Detectar errores en el flujo.
fclose()	Cerrar el archivo.
sizeof	Conocer el tamaño en bytes de un tipo de dato.
errno	Consultar un código de error informado por determinadas operaciones.
_mkdir()	Crear una carpeta en Windows.

Dos aclaraciones técnicas: sizeof es un operador, no una función, y errno tampoco es una función. Además, guardar estructuras completas mediante fwrite() es práctico para esta primera experiencia, pero el formato puede depender del compilador y de cómo se organizan las estructuras en memoria.

6. ¿Qué deberían comprender los estudiantes?

El concepto central no es memorizar todas las funciones, sino comprender tres ideas:

1. Los archivos permiten conservar información cuando finaliza el programa, a diferencia de las variables y los arrays que usamos en memoria RAM.
2. fread() y fwrite() trabajan con direcciones de memoria y tamaños en bytes. Por eso podemos utilizarlas tanto con variables individuales como con arrays de estructuras.

3. La lectura debe ser compatible con la escritura. Si guardamos primero un entero y después tres estructuras, al recuperar los datos debemos leerlos exactamente en ese mismo orden.

Como actividad práctica, les propondría ejecutar el CRUD, crear tres usuarios, cerrar el programa y volver a abrirlo. Después, modificar el archivo usuarios.dat con un editor hexadecimal (trabajando sobre una copia) y observar qué ocurre cuando la cantidad inicial de usuarios deja de coincidir con los registros almacenados. Es una forma concreta de comprender por qué son necesarias las validaciones de lectura.